# NukMiris v0.8.1 — Independent Greenfield Architecture Challenge

## Deliverable 2 of 5 — Greenfield Architecture Proposal

*Independent architect submission · 26 October 2026 · Design only. Requirement IDs, findings (F-n), proposed targets (T-n) and provider spikes (S-n) refer to Deliverable 1. Decisions (ADR-n) are justified in Deliverable 3.*

## 1. The design in one page

**Shape.** A TypeScript modular monolith built from one codebase and run as two container roles, `web` and `worker`, over one managed PostgreSQL. Postgres is more than storage. It is the inbox, the conversation-state store, the operation ledger, the outbox, the reminder schedule, the reservation arbiter and the audit trail. That is what lets “accepted”, “processed” and “queued reply” commit together (I04, I05, I12).

**Appointments.** Google Calendar stays the truth for whether an appointment exists, when it is and which version it is. Every Calendar write is bracketed by a durable intent before the call and a verified outcome after it. Creation is made idempotent with client-assigned event identifiers **[verify S-1]**. Ownership is carried in private extended properties on every event.

**Messages.** WhatsApp sends cannot be made idempotent, so each attempt is recorded before submission and is never retried blindly. Receipts correlate to the exact attempt through an opaque callback value **[verify S-6]** and are merged by a monotonic rule, so arrival order does not matter (I09).

**Isolation.** The tenant is derived once from the authenticated receiving channel, then enforced by composite foreign keys and row-level security. Application code cannot forget a `WHERE business_id = …` because the database refuses the query (I01, W02).

## Eight rules every component obeys

| # | Rule |
|---|---|
| R1 | One database transaction decides internal truth: accepted input, new state, queued replies and new operations commit together. |
| R2 | External effects happen outside transactions, bracketed by a durable intent before and a durable outcome after. |
| R3 | Every external call returns *definite* or *uncertain*. Unknown is uncertain. Uncertain is retried only if the call is idempotent by construction. |
| R4 | Google is truth for appointments. The database is truth for execution history. Neither overrides the other's domain. |
| R5 | Tenant scope is structural: derived once from the authenticated channel and enforced by the data layer. |
| R6 | Time is stored as instants. Business-local dates are derived in exactly one time module. |
| R7 | The core is pure: decisions are functions from state, input and context to a new state plus declared effects. |
| R8 | Recovery is a normal code path. Every restart runs the same reconcile logic that incident recovery uses. |

# 2. Recommended technology stack

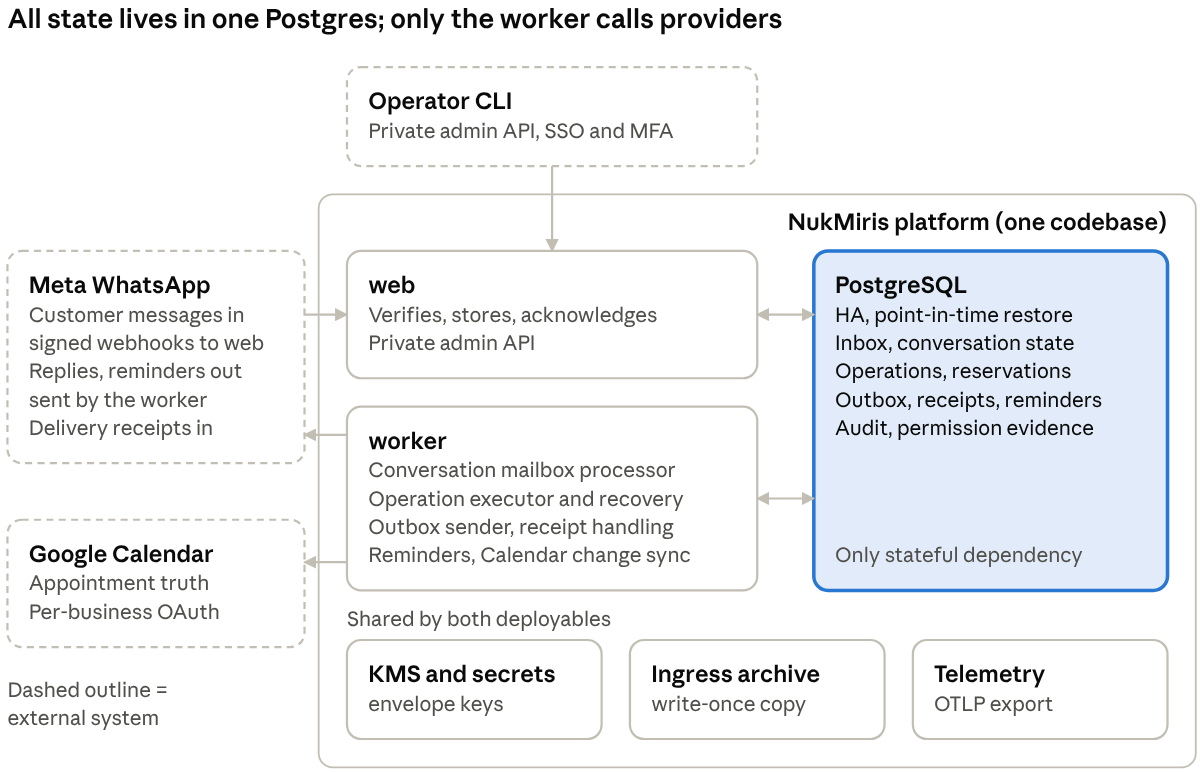
| Layer | Choice | Reason | Main alternative rejected |
|---|---|---|---|
| Language and runtime | TypeScript (strict) on the current Node.js LTS | Continuity with the POC's ecosystem, official Google client, JSON-heavy provider payloads, one set of schema types end to end. | Go: stronger concurrency and a single binary, but an unfamiliar language for a small team (ADR-002). |
| HTTP | Fastify | Raw request bytes are available for signature verification; built-in schema validation. | Express: workable, but raw-body handling is easier to get wrong. |
| Validation | Zod at every boundary | Provider payloads, JSON columns and configuration are parsed, never trusted. | Hand-written guards. |
| Database | Managed PostgreSQL 16+ with HA and point-in-time recovery; `btree_gist` extension | Transactions, row-level security, exclusion constraints, `SKIP LOCKED` (ADR-003). | SQLite (single writer, no HA); DynamoDB (no constraint arbiter). |
| Data access | Kysely typed SQL builder with plain-SQL migrations | RLS, exclusion constraints and per-transaction settings need first-class SQL. | ORMs such as Prisma: weaker support for those features. |
| Async execution | Leases and fencing on Postgres tables; a thin in-house layer | The domain needs per-conversation order and uncertain-outcome states that generic queues do not model (ADR-004). | SQS, RabbitMQ, Kafka, Redis queues, Temporal, pg-boss, Graphile Worker. |

| Layer | Choice | Reason | Main alternative rejected |
|---|---|---|---|
| Time | Temporal API (polyfill) or Luxon, chosen in spike S-11. No raw `Date` arithmetic. | Explicit handling of nonexistent and repeated local times. | `Date` plus ad hoc offsets. |
| Phone handling | libphonenumber-js for display only | The identity key stays the provider `wa_id` (F-06). | E.164 as the key. |
| Cryptography | Node `crypto` (HMAC-SHA-256, AES-256-GCM) with cloud KMS envelope keys | Standard primitives; no custom schemes. | Self-managed key files. |
| Observability | OpenTelemetry SDK, pino JSON logs with redaction, OTLP export to a managed backend | Vendor-neutral. | Vendor-specific agents. |
| Testing | Vitest, Testcontainers (real Postgres), fast-check, provider simulators, crash-point harness | See §17. | Mocked databases. |
| Delivery | Minimal container image; OpenTofu or Terraform; CI with SBOM, image signing and dependency review | Reproducible, reviewable infrastructure. | Hand-configured hosts. |
| Hosting | Managed containers plus managed Postgres, one region | See §14 (ADR-015). | Kubernetes; a single VM. |

The POC's Node and SQLite choices are not binding (they are C-class). Staying in TypeScript is a team-continuity decision, not a technical claim that it is the only suitable language.

# 3. System context and runtime topology

*Figure 1 — Runtime topology: who talks to whom, and where state lives.*




runtime topology · web and worker over one Postgres, two providers, shared services

Inbound traffic reaches only `web`. Every provider call originates in `worker`. The two meet only in Postgres.

| Component | Responsibility | Deliberately does not |
|---|---|---|
| `web` | Verifies webhook signatures, stores accepted events, acknowledges. Serves the private admin API. | Call Google or Meta. Hold per-business credentials. |
| `worker` | Runs every background loop (§4.2). The only process that decrypts per-business tokens and calls providers. | Accept inbound internet traffic. |

| Component | Responsibility | Deliberately does not |
|---|---|---|
| PostgreSQL | Sole stateful dependency. Holds inbox, state, ledger, outbox, schedule, evidence and audit. | Hold credentials in plaintext. |
| KMS and secrets manager | Key-encryption keys for per-business tokens; platform secrets (Meta app secret, Google OAuth client secret). | Appear in application logs or the database dump. |
| Ingress archive | Write-once copy of raw signed webhook bodies, used for disaster recovery and forensics (§16). | Serve as the primary acceptance store. |
| Telemetry backend | Receives logs, metrics and traces over OTLP. | Receive message text or raw phone numbers. |
| Operator CLI | Thin client for the private admin API with SSO and MFA. | Reach the database directly in normal operation. |

Network posture: the public internet reaches only the webhook path. The worker’s egress is limited to the Meta Graph host, Google API hosts, KMS and the telemetry endpoint. The database has no public address.

# 4. Application architecture

## 4.1 Layers and modules (hexagonal, dependency rule enforced in CI)

| Layer | Modules | Rule |
|---|---|---|
| `domain` (pure) | `conversation` (state types, reducer, intent and date grammar, copy keys); `scheduling` (availability engine, slot labels, DST policy); `operations` (state machine, outcome types, classification rules); `reminders` (schedule calculator, eligibility, attendance machine); `messaging` (attempt machine, receipt merge, service-window rules) | No I/O, no clock, no randomness. Imports nothing from other layers. |
| `application` | IngestInbound, ProcessConversation, ExecuteOperation, ReconcileOperation, SendOutbound, ReminderTick, SyncCalendar, RecordPermission, OperatorActions | Orchestrates transactions and ports. Owns transaction boundaries. |
| `adapters` | `postgres` (tenant-scoped session wrapper, repositories), `google`, `meta`, `kms`, `clock`, `ids` | Implements ports. Classifies every provider result as applied, definitely rejected or uncertain. |
| `interfaces` | `http` (webhook, admin API), `worker` (loops), `cli` | No business rules. |

Boundaries are enforced by lint rules (for example dependency-cruiser) so a violation fails CI. The pure `domain` layer is what makes exhaustive property testing and deterministic crash simulation affordable.

## 4.2 Worker loops

| Loop | Claims | Trigger | Notes |
|---|---|---|---|
| Mailbox | Conversations with unprocessed events | Wake-up notification plus 1 s poll fallback | Single writer per conversation via lease. |

| Loop | Claims | Trigger | Notes |
|---|---|---|---|
| Operation executor | Operations that are `unsubmitted`, or `in_flight` with an expired lease | 1 s poll | Runs the saga in §8. |
| Reconciler | `uncertain` operations past the settle window; send attempts with unresolved receipts | 30 s poll | Read-only proof first (§9). |
| Outbox sender | Next sendable message per conversation | 1 s poll | Strict order; bounded receipt-wait (F-05). |
| Reminder scheduler | Due, unexpired occurrences | 5 s poll | Comfortable against the 15-minute allowance (T-5). |
| Calendar sync | Businesses with live reminders or upcoming appointments | About every 2 min per business, backing off when idle | `updatedMin` polling plus preflight reads (§7.5). |
| Housekeeping | Expired offers, retention, archive export, invariant sentinels | Minutes | Sentinels in §15. |

Intervals are tunable defaults, not commitments. Notifications are only an optimisation: polling is authoritative, so a lost wake-up costs latency, never correctness.

### 4.3 The lease-and-fence pattern (used by every loop)

Each claimable row carries `lease_owner`, `lease_until` and a monotonically increasing `fence`. A claim is one statement: select ready rows with `FOR UPDATE SKIP LOCKED`, set the lease, increment `fence`, return the rows. Every commit that depends on the claim includes `WHERE fence = :claimed_fence`. A worker heartbeats its lease and abandons the work if the fence has moved. A stale worker that wakes after a pause therefore cannot overwrite a successor's result or release its protection (I06). The database clock is the single time authority for leases and due comparisons; application clocks never decide whether something is due.

# 5. Inbound pipeline

## 5.1 Webhook ingress ( `web` )

1. **Authenticate the exact bytes.** Compute HMAC-SHA-256 over the raw body with the Meta app secret and compare in constant time. A failure stores nothing and returns 401 (W01). The challenge handshake is handled separately.
2. **Parse** with Zod. Unknown shapes are stored as raw batches and flagged, not discarded.
3. **Resolve the tenant.** The receiving phone-number identifier is looked up in `business_channels` through a narrow `SECURITY DEFINER` function. An unknown channel is written to `quarantined_ingress` with no tenant, an alert is raised and 200 is returned so the provider stops retrying. There is no default tenant (W02).
4. **Accept in one transaction.** Insert the raw batch (body, signature, receipt time). For each message or status, insert into `inbound_events` with `ON CONFLICT (provider_event_key) DO NOTHING`. On conflict, compare a SHA-256 content hash. A mismatch is written to `integrity_conflicts` and alerted, and the original is never substituted (I04). Allocate a per-conversation sequence number by incrementing `conversations.next_seq` under the row lock.
5. **Acknowledge only after commit.** A database failure returns 503 so Meta retries **[verify S-7]**.
6. **Archive best-effort** after commit: write the raw body to the ingress archive. This is a recovery aid, not part of acceptance.

The provider event key is the message ID for messages and the message ID plus status plus provider timestamp for statuses.

## 5.2 Why a per-conversation counter

Global identity values are not commit-ordered: two concurrent transactions can commit out of sequence. A counter row incremented under a row lock is. The lock is held until commit, so for one conversation the sequence order equals the commit order. That gives the accepted, per-conversation order I05 requires without a global lock.

## 5.3 Mailbox processing ( `worker` )

For each leased conversation, events are processed strictly in sequence. Each one runs three phases:

- **Phase A, read transaction:** load conversation state, offers, customer and business configuration.
- **Phase B, no transaction open:** any read-only provider calls the decision needs (for example availability). Nothing here has a side effect, so repeating it is safe.
- **Phase C, write transaction:** verify `state_version` and `fence`, then in one commit store the reducer's new state and offers, the ordered reply group in the outbox, any new operation, and the event's `processed` mark.

A crash anywhere before phase C commits leaves the event unprocessed and the replay produces the same decision. There is no Calendar write and no message send inside this path, which is why replay cannot duplicate an effect (I02, I05).

```
sequenceDiagram
  participant M as Meta WhatsApp
  participant W as web
  participant DB as PostgreSQL
  participant K as worker mailbox
  participant G as Google Calendar
  M->>W: POST webhook, signed raw bytes
  W->>W: verify HMAC over raw body
  W->>DB: tx: store batch, dedupe events, assign sequence
  DB-->>W: commit ok
  W-->>M: 200
  K->>DB: claim conversation lease, fence n
  K->>DB: read state, offers, business config
  K->>G: read-only availability, no tx open
  K->>DB: tx: new state, reply group, operation, event processed, fence n
  Note over K,DB: a crash before this commit leaves the event unprocessed and
replay is safe
```

## 5.4 Poison events and unsupported input

An event whose processing fails repeatedly (default three attempts with backoff) is parked as `failed` with its error, a review item is opened and an alert fires. The conversation then continues, because a permanently stuck conversation is worse than one ignored message; the customer can resend or send Menu. Media, reactions, locations and other unsupported types are not downloaded or interpreted: the reducer produces a short Spanish reply that only text and buttons are supported.

# 6. Conversation state design

## 6.1 State as a typed union

The baseline's semantic transitions (Section 3) are implemented one-to-one. State is a discriminated union persisted as validated JSON on the conversation row together with a `state_version`. JSON is appropriate here because the state is transient (minutes) and fully re-derivable; the durable facts (customers, operations, appointments, evidence) are relational.

| Variant | Carries | Lifetime |
|---|---|---|
| Main | Current menu offer set | Until replaced |
| Browse | Purpose (new or replace-appointment), stage, search cursor, selected date, offer set | 15 min, renewed on each new view |
| NeedName | Pending selected instant, offer provenance, attempt count | 15 min |
| Manage | Offer set of owned upcoming appointments | 15 min |
| Clarify | Candidate intents | 15 min |
| Human | Entered-at | Until Menu |
| OperationResult | Operation reference | Until Menu or superseded view; the operation itself is durable |
| ExpiredView | Why it expired | Until the customer acts |

## 6.2 Offers: server-side authority for every action

Every interactive message is rendered from an `offer_set` row that holds `offers`. A button or list-row identifier is an opaque 128-bit random token. The token resolves to a stored action payload (select instant, select appointment, more, choose date, confirm attendance and so on).

A token is honoured only if all of these hold: it belongs to the same business and conversation as the inbound message, its set has not expired or been superseded, and it has not been consumed (for single-use actions). Anything else lands in ExpiredView with

recovery guidance and **no appointment or customer effect** (I03). Labels such as `mié 23 jun 10:00` are rendered text only and never identifiers; full instants are stored in the payload (W09).

Durable operation actions (status, recorded-operation repeat) use `operation_actions` bound to an operation ID with no 15-minute coupling, so replacing a view cannot orphan a recorded operation. Reminder response actions are bound to the occurrence and current schedule identity and stay valid until the appointment starts (§11).

## 6.3 Pure reducer

The core is one function: `decide(state, input, context) → { state', effects[] }`. Effects are declared data: `SendMessage`, `CreateOperation`, `RegisterCustomer`, `EnterHandoff`, `RecordWithdrawal` and similar. The application layer turns them into rows inside the phase C transaction. Because `decide` is pure, three properties are tested exhaustively and by generated inputs:

- every state and input pair yields a valid state (no unreachable or illegal transitions);
- browsing, date selection and recognition alone never emit a mutating effect (C04, W06, W07);
- Menu never emits an effect that touches a recorded operation (Section 3, item 5).

## 6.4 Name entry, registration and submission

Name validation is a pure replaceable function (U-NAME). Original and normalized values are stored separately. Only the exact global `menu` command overrides name entry; other words are validated as names (proposed resolution in Deliverable 1). A valid name registers the scoped customer **and** creates the booking operation for the previously selected instant in the same transaction, so a crash cannot leave a registered customer with a lost selection, or a selection executed for an unregistered one (W03). Nothing is reserved while the name is pending; the executor rechecks availability (F-09).

## 6.5 Human mode

Ordinary text and interactive actions in Human mode are stored as original messages under that business and customer and never reach the appointment reducer. Menu closes the mode. An explicit withdrawal (STOP and the supported equivalents) is recognised before mode routing, recorded as permission evidence and honoured without cancelling any appointment (W05, R09). The handoff queue is exposed read-only to operators (F-11).

### 6.6 Intent and date grammar

Both are deterministic, table-driven pure modules with a fixture suite and generated tests, not NLU. Recognition returns candidate intents or a parsed civil date, never an effect. Normalization (case, accents, whitespace) is applied to a copy; originals are kept. The documented date forms resolve against the business's civil date, with next-occurrence semantics and no overflow (W08). A bare-weekday fallback to next week happens only after a successful availability check shows no remaining starts today.

### 6.7 Copy catalog and WhatsApp limits

All customer text lives in a typed es-MX catalog keyed by message identifier. The exactly specified strings (for example the no-appointments message and the reminder bodies) are snapshot-tested. Outgoing payloads are built through a typed builder that refuses to construct an invalid message: at most three reply buttons with titles up to 20 characters, at most ten list rows, template buttons within their own limit (W09, R10).

## 7. Google Calendar integration design

### 7.1 Authorization and calendar selection

Each business authorizes NukMiris through its own OAuth consent, with offline access and the narrowest scope that permits create, patch, delete and free/busy **[verify S-5]**. The platform OAuth client secret lives in the secrets manager.

Consent starts only from an operator action that creates a one-time `consent_request`: random `state`, a server-held PKCE verifier, a 10-minute expiry, and a binding to the business and the operator. The callback validates `state` and PKCE, exchanges the code, then **verifies the granted scopes and that the chosen Calendar is reachable with the required role** using a read-only probe. Only then is the stored credential replaced, in one transaction. A failed or abandoned consent leaves the working authorization untouched (I11).

The Calendar is chosen explicitly from a discovery listing. There is no `primary` fallback. A unique index allows a Calendar to map to at most one active business (F-04). Changing the mapping invalidates open offers, restricts in-flight operations to reconciliation, and requires re-verification (C01).

## 7.2 Event identity and ownership

| Field | Value | Purpose |
|---|---|---|
| Event `id` | Client-assigned 128-bit random value, base32hex, allocated when the operation is created and stored on it | Idempotent create; exact lookup during reconciliation **[verify S-1]** |
| Private extended property `nm_biz` | Business ID | Ownership predicate; filter for lookups and sync |
| `nm_cust` | Customer ID (not the phone) | Per-customer lookup (C06) |
| `nm_op` | ID of the creating operation | Proves a given operation's insert |
| `nm_sched` | Current schedule identity | Reminder and attendance targeting (R07) |
| `nm_chg` | ID of the last change operation | Proves a given move |
| `nm_att` | ID of the attendance operation | Proves a given confirmation |
| `nm_sfx` | Marker that the title suffix is app-owned | Safe removal on reset (R05) |
| `summary` and `description` | `{Customer Full Name} — Cita`; Cliente, WhatsApp, `Estado: Reservada`, `Origen: NukMiris WhatsApp` | Owner-visible content (C05). No service information. |

**Managed by NukMiris for this business** means: the event is on the mapped Calendar *and* `nm_biz` equals the acting business. Every read of an event by any operation applies this predicate before acting (I01, C06). Untracked and manual events are never treated as appointments, yet still block time through availability.

## 7.3 Availability

- **Pure engine.** `computeSlots(config, busy, now, window)` applies weekday hours, breaks, duration, start interval and buffer. Intervals are half-open, so touching is not overlapping. Nonexistent and repeated local times are excluded (F-15). Labels are produced separately from instants (C01, W09).

- **Provider reads.** Free/busy queries over windows of about 31 days **[verify S-4]**. Discovery reads the first window and extends window by window until three available dates are found or 365 civil days in the business timezone are covered. The worst case is about twelve calls, bounded by a per-request deadline and a per-business circuit breaker. A directly selected date is queried on its own, regardless of earlier suggestions (C02).
- **Result type.** `Available(slots)` or `Unavailable(reason)`. Any error, timeout, malformed body or per-calendar error entry is `Unavailable`, never empty (C03). The customer sees that the agenda cannot be checked right now, with a retry action.
- **Caching.** A short-lived cache (default 30 seconds) is keyed by Calendar and window and is used for browsing only. It is invalidated by our own writes. It is never used for the final recheck, so a stale browse can offer a slot that has gone, and the recheck catches it.
- **Final recheck.** A fresh, uncached read of the exact interval plus buffer, ignoring the moving appointment's own event during a reschedule (C04, C07).

## 7.4 Lookup of a customer's appointments

A filtered event list on `nm_biz` and `nm_cust`, from now, with a restricted field set **[verify S-3]**. Past, cancelled, recurring, mismatched-owner and untracked events are excluded. A provider failure is an error state, never an empty list (C06). Results refresh the `appointments` projection and its `observed_at`. The projection is a cache of Google, not a second truth (R4).

## 7.5 Detecting manual changes

- **Sync loop.** Per business, list events with `updatedMin` set to the last cursor minus a 5-minute overlap, deleted events included, filtered to `nm_biz`. Upserts are idempotent. The cursor advances only after the batch commits. Detected changes drive the rules in §11.6.
- **Preflight reads.** Before every reminder submission the event is read again (§11.3), so correctness never depends on poll lag.
- **Push channels** are a later optimisation. Polling needs no public callback and no channel renewal, which is the simpler thing to operate.

### 7.6 Result classification and quota

| Signal | Class | Action |
|---|---|---|
| Success with a verified body | Applied | Commit |
| Conflict on insert with our identifier | Possibly applied earlier | Read the event, verify `nm_op` and times, then Applied |
| Precondition failure on a conditional write | Definitely rejected (stale) | Re-read; re-present or fail as stale |
| Rate-limit response | Definitely not processed | Back off with jitter; the same request is safe |
| Token refresh failure | Definitely not processed (never reached the API) | Mark credential failed (§13.4) |
| Not found or gone on delete | Cancellation likely | Read to confirm the event was ours and is cancelled |
| Timeout, reset after send, 5xx, malformed body | Uncertain | Reconcile (§9) |

The exact mapping per endpoint is a spike deliverable (S-1 to S-4). Calls pass a per-business token bucket and a global ceiling. Retries use exponential backoff with jitter and a bounded attempt count and deadline (I09).

## 8. Booking, cancellation and rescheduling

### 8.1 Operation ledger

An `operations` row records the business, customer, conversation, kind (book, move, cancel, attend), the source offer (unique, so one offer yields at most one operation), the requested interval or intervals, the pre-allocated event identifier, the configuration version, status, lease and fence, and the outcome detail. `operation_events` is an append-only history of every transition and every piece of provider evidence.

Status model, mapping to the baseline's Section 4: `unsubmitted → claimed → in_flight →` one of `verified_applied`, `definitely_rejected`, `uncertain`; then `uncertain → verified_applied` or `review`. `abandoned` covers decline, supersession and expiry before submission.

## 8.2 Reservations serialize our own writers

`calendar_reservations` holds `calendar_id`, `appointment_key`, `operation_id`, `during` (a half-open time range that includes the buffer) and `state` (`held` or `committed`). One constraint does the arbitration:

`EXCLUDE USING gist (calendar_id WITH =, during WITH &&, appointment_key WITH <>) WHERE (state IN ('held','committed'))` **[verify S-10]**

Two rows conflict only when they share a Calendar, overlap in time and belong to *different* appointments. Because the key is the event identifier, a move's new interval may overlap its own old reservation but never another appointment's. With a zero buffer, adjacent appointments touch without conflict (C01). Capacity is one per Calendar (F-01).

The reservation is inserted in the same transaction that creates the operation. A violation is a definite local rejection ("slot just taken") and the customer gets fresh options. Release and commit transitions are fence-checked, so a stale worker cannot release a successor's protection (I06). No in-process lock is involved, so any number of instances is safe.

## 8.3 Booking saga

1. **Selection (mailbox transaction).** Validate the offer. A new customer supplies a name first (W03). Insert the operation, the `held` reservation and the OperationResult state. No extra confirmation question is asked (C04).
2. **Claim.** The executor claims the operation and records `in_flight` in a transaction *before* any Calendar call.
3. **Fresh recheck.** `Unavailable` ends the operation as definitely rejected (agenda could not be verified), releases the reservation and offers a retry. A busy interval ends it as definitely rejected (slot taken) with fresh options.
4. **Submission marker.** A separate small transaction records `submission_started_at` immediately before the HTTP call.
5. **Insert** with the pre-allocated identifier and the ownership properties.

6. **Verify** identifier, times, properties and status. A conflict response is read and verified instead of treated as failure. A timeout, 5xx or malformed body marks the operation `uncertain`.
7. **Commit (one fence-checked transaction).** Operation `verified_applied`; reservation `committed`; appointment projection upserted; reminder enrollment per mode (§11.1); reply group queued. The reply carries the reminder promise only when mode is live, permission is effective and at least one future occurrence exists (R11). A failed reminder update rolls back the whole commit, so protection is never released on a false finalization (I08).
8. **Overlap audit.** A fresh read around the interval looks for non-NukMiris events that now overlap. These are flagged to the review queue and never auto-cancelled (U-EXTERNAL).

```
sequenceDiagram
  participant MB as mailbox
  participant DB as PostgreSQL
  participant EX as operation executor
  participant G as Google Calendar
  MB->>DB: tx: operation, held reservation, event id allocated
  EX->>DB: claim, set in_flight, fence n
  EX->>G: fresh read of interval and buffer
  EX->>DB: record submission started
  EX->>G: insert event with allocated id and ownership properties
  alt response verified
    EX->>DB: tx: applied, reservation committed, appointment, reminders,
reply group
  else timeout, 5xx or malformed
    EX->>DB: mark uncertain
    EX->>G: after settle window, read by id, then same-id insert only if
absent
  end
```

## 8.4 Crash points and recovery

| Crash point | State on restart | Recovery | Customer sees |
|---|---|---|---|
| After selection commit, before claim | `unsubmitted`, reservation held | Executor claims normally | Nothing lost |

| Crash point | State on restart | Recovery | Customer sees |
|---|---|---|---|
| After claim, before `in_flight` | Lease expires | Reclaim | Nothing lost |
| After `in_flight`, before submission marker | No marker, so no Calendar call was made | Re-run from the fresh recheck | Nothing lost |
| After marker, before the response is processed | `uncertain` | Read by event ID. Found and verified: commit, no second insert. Not found: wait for the settle window, then recheck and retry the *same-ID* insert | Result delayed, never duplicated |
| After provider success, before commit | Same as above; the event exists | Read finds it; commit | Result delayed |
| After commit, before reply is sent | Reply is in the outbox | Sender sends once | Nothing lost |
| After reply submitted, before outcome recorded | Attempt `submitting` | Marked uncertain; resolved by receipt; never resent (§10) | At most one message |

The submission marker is what separates “definitely never sent” from “possibly sent”. Without it, every crash would cost a full settle window.

## 8.5 Cancellation

Selecting an owned appointment creates the cancel operation. The executor re-reads the event (ownership predicate, current times, ETag). A mismatch is a definite stale rejection and the list is refreshed. Otherwise it deletes with an `If-Match` precondition **[verify S-2]**, confirms by read, and commits in one transaction: operation applied, appointment `cancelled`, reservation released, remaining reminder occurrences and attendance actions invalidated, reply queued. An uncertain delete is proven by a matching cancelled

event or a not-found or gone response on an event that was ours. Appointment state is monotonic (`cancelled` is terminal) and every commit is fence-checked, so late callbacks cannot resurrect it or release a newer operation's protection (C08).

### 8.6 Rescheduling

One appointment goes straight to availability; several first need a selection. Choosing the replacement time creates the move operation. The executor reads the source event, checks the target interval ignoring that event, inserts a second reservation under the same appointment key (the original stays), then patches the *same event* with `If-Match`: new start and end, `nm_chg` set to this operation, `nm_sched` set to a new schedule identity, and any app-owned title suffix removed. Customer details and unrelated fields are untouched (C07).

The commit swaps the reservations, updates the projection, invalidates old occurrences and enrolls only still-future ones, and resets attendance to pending (R07). Failure or abandonment never touches the original reservation. An uncertain patch is proven by reading the event: `nm_chg` equals this operation and the times equal the target. Protection stays until the settle window has passed.

### 8.7 Attendance confirmation

The same pattern: append `— Confirmada` once with `If-Match`, set `nm_att`, verify by read. An unknown outcome is never acknowledged as success and never patched twice (R05).

## 9. Idempotency and uncertainty handling

### 9.1 Idempotency catalogue

| Layer | Key | Mechanism | A duplicate does |
|---|---|---|---|
| Webhook ingress | Provider event key | Unique constraint plus content hash | Nothing; a changed body is an integrity conflict |
| Event processing | Event ID | Effects and the `processed` mark share one transaction | Replays to the same decision |

| Layer | Key | Mechanism | A duplicate does |
|---|---|---|---|
| Offer use | `offer_id` on the operation | One operation per offer | Returns the recorded status (I02) |
| Calendar create | Client event ID | Provider uniqueness plus read-verify | A conflict is verified, not duplicated |
| Calendar change, cancel, attend | ETag precondition plus change identity in properties | Conditional write; property match on read | Verified, not repeated |
| Outbound send | Attempt ID recorded before the call | No provider-side idempotency; uncertain is never retried | Operator-only retry (U-RETRY) |
| Reminder | Appointment, schedule, kind | Unique constraint | No second row |
| Receipt | Message ID, status, provider timestamp | Monotonic merge (§10.3) | Idempotent |
| Permission evidence | Customer plus evidence reference | Reuse with conflicting content is rejected | Explicit conflict |
| Operator action | Action, target, request ID | Dedupe plus audit | Recorded once |

## 9.2 Outcome policy

- **P1.** Unknown is uncertain. Nothing is ever assumed to have failed.
- **P2.** An uncertain Calendar create is reconciled by event ID. After the settle window, a *same-ID* retry is permitted because it is idempotent (F-03).
- **P3.** An uncertain Calendar change is reconciled by the identity stored in the event's properties. It is re-sent only if proven not applied and the settle window has passed.
- **P4.** An uncertain send is never retried automatically. It is resolved by receipt correlation or sent to review. An expired uncertain send becomes terminal and is retained.
- **P5.** A customer is never told success for an uncertain operation. They are told it is being confirmed, and the final outcome follows if the service window allows (F-08).

- **P6.** Reconciliation is read-only, except the same-ID insert retry. It is repeatable and never forces a release, send or Calendar write that was not already authorized.

The **settle window** (default 10 minutes, T-6) covers the chance that a provider finishes processing after our response was lost. It is configuration, not an invariant, and must be validated against observed provider latency.

# 10. Outbound messaging

## 10.1 Outbox and attempts

`outbound_messages` holds the intent: business, conversation or reminder occurrence, group, sequence within the conversation, type, payload reference, `expires_at` and ordering policy. `outbound_attempts` holds each submission: attempt ID, state, provider message ID, classification and timestamps. Normally a message has one attempt. An operator retry creates a new linked attempt.

**Expiry.** An ordinary reply expires at the earlier of the service-window end (last customer message plus 24 hours) and, for prompts, its session expiry. The rule is revalidated at send **[verify S-8]**. An expired unsent message is retained as history and never blocks fresh work (I13).

**Order.** Strict per conversation. The next message is eligible when the previous one is terminal or certain, or when its receipt-wait (default 5 minutes) has elapsed (F-05).

**Send path.** Claim; check expiry, credential health and, for reminders, permission and template; commit `submitting`; call the provider with an opaque callback value equal to the attempt ID **[verify S-6]**; record the outcome in a transaction.

```
stateDiagram-v2
  [*] --> Prepared
  Prepared --> Submitting: marker committed before the call
  Submitting --> Accepted: provider message id returned
  Submitting --> Rejected: definite refusal
  Submitting --> Uncertain: timeout, 5xx or crash
  Uncertain --> Accepted: receipt carries the attempt id
  Uncertain --> ExpiredUncertain: window or session ended
  Uncertain --> Closed: operator retry creates a new linked attempt
  Accepted --> Sent
  Sent --> Delivered
  Delivered --> Read
  Rejected --> [*]
  ExpiredUncertain --> [*]
  Closed --> [*]
```

## 10.2 Classification of Meta send results

| Signal | Class | Action |
|---|---|---|
| 2xx with a message ID | Accepted | Record ID |
| Validation, policy, window or template error | Definitely rejected, permanent | Record reason; no retry |
| Invalid or expired token | Definitely rejected (credential) | Pause that business's sender; the retry after rotation is safe |
| Throttling or 429 | Definitely not processed | Back off; safe retry |
| DNS or connect failure before bytes sent | Definitely not sent | Safe retry |
| Timeout after send, reset, 5xx, malformed body | Uncertain | Await receipt; never auto-retry |

The exact error-code mapping is fixed in spike S-6. Anything not positively classified is uncertain.

### 10.3 Receipts: a monotonic merge

`message_receipts` is append-only: business, channel, provider message ID, status, provider timestamp, received-at, raw reference and a nullable correlated attempt. Delivery evidence ranks `accepted < sent < delivered < read`. A `failed` receipt is recorded but cannot lower a rank at or above `delivered`. The effective state is the highest rank, so the merge is commutative and idempotent and any arrival order yields the same answer (I09).

Correlation uses the attempt ID, then checks that business, sending channel, recipient `wa_id` and message ID all match that attempt. Anything that does not match is stored as an *orphan* with no effect on any attempt. A receipt therefore can never resolve another customer's uncertain send. A receipt that carries an uncertain attempt's ID upgrades it to `accepted` with proof. Receipt time is kept distinct from the customer's device or click time (O01).

### 10.4 Why uncertain sends are not retried

A second identical message is customer-visible noise, and for reminders it is also a compliance risk. The cost of not retrying is one missing message that the customer can recover by writing again. For reminders the 15-minute allowance usually closes the question anyway.

## 11. Reminders and attendance

### 11.1 Eligibility and enrollment

At a verified booking commit, in the same transaction, a reminder-enabled business gets occurrences computed with **instant** arithmetic: due at start minus 168 h, 24 h, 22 h (only if the 24 h request was enrolled) and 2 h. An occurrence is enrolled only if its due instant is strictly in the future at commit (R02). Under U-ACTIVATION parity, intent creation must be at or after activation; both the intent and commit timestamps are stored so the other policy needs no migration. An uncertain booking enrolls nothing. Reconciliation enrolls an eligible finalized booking later and sends nothing at that moment (R01).

`reminder_occurrences` holds appointment, `schedule_id`, kind, `due_at`, `expires_at` (the earlier of due plus 15 minutes and appointment start), state and attempt. A unique key on appointment, schedule and kind guarantees one applicable occurrence.

```
stateDiagram-v2
  [*] --> Future: enrolled at verified commit
  Future --> Due: due instant reached
  Future --> Cancelled: moved, cancelled or manually changed
  Due --> Cancelled: moved, cancelled or manually changed
  Due --> Skipped: late beyond 15 min, appointment started, or checks failed
until the allowance ended
  Due --> Previewed: dry-run mode
  Due --> Submitting: preflight passed, marker committed
  Submitting --> Accepted
  Submitting --> Rejected
  Submitting --> Uncertain: crash or unknown outcome
```

## 11.2 Schedule identity

`schedule_id` rotates on every move and on every manual unconfirmed move. In the same transaction, remaining occurrences of the old schedule are cancelled. Action tokens carry occurrence and schedule identity and both are checked on use, so a delayed reply can never confirm the wrong schedule (R07, R12).

## 11.3 Scheduler and send preflight

The scheduler claims due, unexpired occurrences with lease and fence. One that is past `expires_at` is marked `skipped(late)` and never replayed after downtime (R03). Before sending, every check below must pass; otherwise the occurrence stays pending until its allowance ends and is then `skipped` with the recorded reason.

1. **Mode.** Off does nothing. Dry-run records `previewed` and permission gaps and sends, requests and mutates nothing. Live continues (R11).
2. **Permission** is effective: evidence exists and no later withdrawal (R09).
3. **Template** is ready for this business, kind, language and TTL. A cached verification is re-checked at send time; Meta remains final authority (R10).
4. **Calendar.** A fresh read shows the event exists, is ours, and has the expected start and schedule. Otherwise §11.6 applies.
5. **Follow-up only.** The parent request has a correlated delivered or read receipt, and no confirmation or change response exists (R04).
6. **Time.** The database clock is at or after due, before `expires_at`, and before appointment start.

Then record the attempt as `submitting`, send with the callback value, record the outcome. A crash leaves `submitting` with an expired lease, which becomes `uncertain` and is never resent (R03). Template TTLs are template metadata checked at readiness, not a per-send field.

## 11.4 Attendance and rescheduling from a reminder

A response targets only the exact current dispatched request: either the quoted message mapped to its attempt, occurrence and current schedule, or a button token bound to occurrence and schedule. A generic “Sí” or “Confirmo” acts only when exactly one candidate exists (U-GENERIC); otherwise the customer is asked to use the specific reminder button. A confirmation is an `attend` operation (§8.7).

A change request opens the availability flow for that exact appointment, suppresses the follow-up and keeps the original reservation and final reminder. If the customer abandons the flow after the browse lifetime, the previous attendance state is restored and the evidence that they responded is kept. A successful replacement resets attendance to pending under a new schedule identity (R06, R07).

## 11.5 Permission evidence and templates

`permission_evidence` is append-only. Each row is a grant or a withdrawal for a scoped customer, with source, wording, evidence reference, recorder, `occurred_at` and `recorded_at`. Effective permission is the latest by `occurred_at`, with withdrawal winning ties and older imports. A grant needs an existing scoped customer and a past or current evidence time. A withdrawal is accepted before registration, held against the `wa_id`. Reuse of an evidence reference with conflicting content is rejected (R09).

`message_templates` records, per business and kind, the name, language, category, status, TTL, a content hash compared with the contract, and `verified_at`. Status and quality notifications update it **[verify S-7]**. Readiness requires all four kinds approved and matching, otherwise live sending is blocked fail-closed (R10, F-14).

## 11.6 Manual change handling

| Observation (sync or preflight) | Action |
|---|---|
| Event deleted or cancelled | Appointment `cancelled`; occurrences and actions invalidated; reservation released |

| Observation (sync or preflight) | Action |
|---|---|
| Time changed, attendance unconfirmed | New `schedule_id`; old occurrences cancelled; new future occurrences created; no message sent about the change |
| Time changed, attendance confirmed | Appointment moves to `review`; occurrences held; review item opened |
| Ownership properties lost, mismatched, or Calendar changed | `review`; sends stopped |
| Converted to a recurring event | Unsupported; `review` |

No completed, attended or no-show workflow exists, and none is added (Section 6).

# 12. Data model and tenant isolation

## 12.1 Core tables

| Table | Purpose | Key invariants |
|---|---|---|
| `businesses` | Identity, IANA timezone, versioned schedule configuration, capability flags, reminder mode and activation time | Configuration is versioned; each operation records the version it used (F-10) |
| `business_channels` | Phone-number identifier and WABA mapped to a business | Phone-number identifier unique among active rows; changes are audited and invalidate sessions |
| `provider_credentials` | Encrypted per-business tokens, versions, status, scopes, `last_verified_at` | Ciphertext bound to business and provider (§13.3) |
| `customers` | Scoped identity: provider `wa_id`, original and normalized name | Unique on business plus `wa_id`; same phone at two businesses is two rows |

| Table | Purpose | Key invariants |
|---|---|---|
| `conversations` | Mailbox: mode, typed state, `state_version`, `next_seq`, lease, fence, last customer message time | One row per business plus `wa_id`; single writer by lease |
| `inbound_batches`, `inbound_events` | Raw signed bodies; deduplicated events with sequence and processing state | Unique provider event key; content hash |
| `offer_sets`, `offers`, `operation_actions` | Server-side action authority (§6.2) | Opaque tokens; scoped to business and conversation |
| `operations`, `operation_events` | Operation ledger and append-only evidence | Unique source offer; status transitions guarded by fence |
| `calendar_reservations` | Interval protection | Exclusion constraint (§8.2) |
| `appointments` | Projection of Google events with `observed_at`, ETag, schedule identity, attendance, state | State is monotonic for terminal values |
| `reminder_occurrences` | The reminder schedule | Unique on appointment, schedule, kind |
| `outbound_messages`, `outbound_attempts`, `message_receipts` | Outbox, submissions, evidence | Attempt marker committed before any send |
| `permission_evidence`, `message_templates` | Compliance data | Append-only evidence; contract hash on templates |
| `review_items` | Work needing a human decision | Typed, scoped, linked to evidence |
| `audit_log` | Operator and system actions | Append-only; no UPDATE or DELETE grant |

| Table | Purpose | Key invariants |
|---|---|---|
| `quarantined_ingress`, `integrity_conflicts` | Untenanted or contradictory ingress | No tenant processing ever reads these |

```
erDiagram
  businesses ||--o{ business_channels : has
  businesses ||--o{ customers : scopes
  businesses ||--o{ conversations : owns
  customers ||--o{ conversations : participates
  conversations ||--o{ inbound_events : receives
  conversations ||--o{ offer_sets : renders
  offer_sets ||--o{ offers : contains
  conversations ||--o{ operations : starts
  operations ||--o{ operation_events : records
  operations ||--o{ calendar_reservations : protects
  customers ||--o{ appointments : books
  appointments ||--o{ reminder_occurrences : schedules
  reminder_occurrences ||--o{ outbound_attempts : sends
  outbound_messages ||--o{ outbound_attempts : tries
  outbound_attempts ||--o{ message_receipts : evidenced_by
  customers ||--o{ permission_evidence : grants
  businesses ||--o{ provider_credentials : authorizes
  businesses ||--o{ message_templates : approves
```

## 12.2 Six layers of isolation

| Layer | Mechanism | What it stops |
|---|---|---|
| L1 Derivation | Tenant comes only from the authenticated receiving channel; unknown channels are quarantined | A customer or payload choosing a tenant (W02) |
| L2 Structure | Every tenant table has `business_id NOT NULL` and composite foreign keys such as `(business_id, customer_id) → customers(business_id, id)` | A row referencing another tenant's row |

| Layer | Mechanism | What it stops |
|---|---|---|
| L3 Row-level security | `ENABLE` and `FORCE ROW LEVEL SECURITY` with a policy on `business_id = current_setting('app.business_id', true)`; the setting is applied with `SET LOCAL` at the start of every transaction; an unset setting matches no rows | A forgotten `WHERE` clause; a query run without tenant context |
| L4 Roles | Separate database roles (below); the runtime role has no `BYPASSRLS` and no DDL | A compromised service reading everything |
| L5 Tokens | Offers and actions are validated against business and conversation; Calendar and credential choice always comes from the business row, never from a token or message | Forged or replayed identifiers (I03) |
| L6 Provider binding | Event ownership predicate ( `nm_biz` ), unique Calendar-to-business mapping, ciphertext bound to business | One tenant's credential or event being used for another |

Workers that claim work across tenants use narrow `SECURITY DEFINER` functions that return only `(business_id, id)`. Processing then opens a tenant-scoped transaction. Platform-wide operator queries run under a separate audited role, never the runtime role.

| Database role | Can | Cannot |
|---|---|---|
| `app_ingress` | Resolve channel to business through one function; insert raw batches and events | Read any tenant data |
| `app_runtime` | Read and write tenant tables under RLS | Bypass RLS; run DDL; touch `audit_log` except insert |
| `ops_readonly` | Read through audited views with masked fields | Write |
| `ops_break_glass` | Cross-tenant access with a recorded reason | Anything unaudited; use is time-boxed |
| `migrator` | DDL, run only from the deploy pipeline | Runtime access |

A schema lint in CI fails the build if any table lacks `business_id` and a policy, unless explicitly allow-listed as platform-level. Connection pooling in transaction mode is compatible because the tenant setting is transaction-scoped **[verify S-10]**.

# 13. Authentication, authorization and credentials

## 13.1 Webhook authentication

HMAC-SHA-256 over raw bytes with the Meta app secret, constant-time comparison. During a secret rotation two secrets are accepted for a bounded period. The verify-token challenge is handled on its own path.

## 13.2 Operator access

The admin API listens on a private address only. Operators authenticate through the organization's SSO with MFA and receive short-lived tokens. Roles: *reader* (masked views), *support* (retry reviewed items, record consent evidence) and *admin* (configuration, credentials, restore). Every mutating call requires a reason, carries a request ID and is written to `audit_log` with the operator's identity. Default output masks the phone (last four digits) and names; revealing needs a reason and is audited. No shared accounts; offboarding means disabling the identity-provider account (F-16). For a team of two or more, risky actions (uncertain-send retry, restore) require a second approver.

## 13.3 Credential storage

Per-business Google refresh tokens and Meta access tokens are encrypted with AES-256-GCM using a data key per credential row, wrapped by a cloud KMS key. The additional authenticated data is `business_id | provider | credential_id`, so ciphertext copied to another tenant or provider will not decrypt. Rows are versioned. Platform secrets (Meta app secret, Google OAuth client secret, KMS key identifiers) live in the secrets manager. Only `worker` can decrypt. Access tokens are cached in memory with their expiry and never logged. A log-redaction test fails the build if a token-shaped string reaches output.

## 13.4 Credential failure and rotation

| Event | Behaviour |
|---|---|
| Google refresh fails with `invalid_grant` or revoked consent | Business flagged `calendar_auth_failed`. Availability returns `Unavailable`. `unsubmitted` operations end as definitely rejected. `uncertain` operations wait and are reconciled once authorization is restored. Alert and review item. Working authorization is never replaced by a failed new consent (I11). |
| Meta token invalid or expired | Sender paused for that business. Attempts end as definitely rejected (credential), so retrying after rotation is safe. Reminders are blocked fail-closed. Alert. |
| Planned token rotation | Add the new version as `next`, prove it with a read-only probe, switch atomically, retire the old version after a grace period. |
| KMS key rotation | New key version wraps new data keys; existing data keys are re-wrapped lazily by a background job. |
| OAuth client or app secret rotation | Dual-secret acceptance during the overlap; tokens remain valid across client-secret rotation. |

### 13.5 Threat summary

| Threat | Primary controls |
|---|---|
| Spoofed or tampered webhook | Signature over raw bytes; payload tenant ignored in favour of channel mapping |
| Replay of a valid webhook | Provider event key dedupe; content-hash conflict detection |
| Forged, guessed or replayed button | 128-bit opaque tokens; scoped and expiry-checked server-side |
| Cross-tenant access | §12.2 layers; negative tests generated for every table |
| Stolen database dump | Credentials are ciphertext bound to tenant; KMS keys are separate |
| Operator misuse | SSO, MFA, roles, reasons, immutable audit, masked defaults |
| Compromised dependency | Lockfile, provenance checks, minimal dependency set, SBOM, signed images, egress allow-list |
| Webhook flooding | Cheap HMAC check before any database work; body-size limit; platform rate limiting at the edge |
| Secrets in logs | Redaction at the logger; tests; no message text or full phone in telemetry |

Media is never fetched, so no user-supplied URL is ever dereferenced.

## 14. Hosting and operational model

**Reference topology** (provider-neutral, region to be fixed after F-12): two `web` instances and two `worker` instances across availability zones on a managed container service with always-on capacity; managed PostgreSQL with automatic multi-AZ failover and point-in-time recovery; an object-storage bucket with retention lock for the ingress archive; KMS; a secrets manager; and a managed telemetry backend. Always-on capacity matters because workers poll; platforms that throttle idle CPU are a poor fit unless configured otherwise.

| Topic | Decision |
|---|---|
| Environments | Local (Compose with simulators); staging (real provider sandboxes: a test WABA number and a test Calendar); production, including a canary business |
| Deployment | Rolling container replacement. Workers drain by ceasing to claim and finishing within their lease. Schema changes follow expand then contract, so the previous image always runs against the current schema. Rollback is redeploying the previous image. |
| Configuration | Environment plus secrets manager; feature flags per business (capability, reminder mode), changed through the audited admin API |
| Scaling | Stateless processes scale horizontally; the database scales vertically first; claim queries scale through `SKIP LOCKED`. Per-business Google budgets and Meta per-number throughput are the external limits. Headroom per T-1. |
| Cost levers | One region, no broker, small instances, sampled traces, no Kubernetes. The managed database and telemetry are the largest recurring items and are the right places to spend. |
| Routine operations | Weekly automated restore drill; monthly credential, template and quota audit; time-zone data refresh through the normal image pipeline |

The baseline notes that the current PC acceptance is not an always-on or live-provider certification (U-OPS). Nothing here assumes the POC's hosting.

# 15. Observability and incident recovery

## 15.1 Telemetry model

One trace follows an inbound event through the mailbox, any operation, the outbox attempt and the receipt, linked by correlation keys: `business_id`, `conversation_id`, `event_id`, `operation_id`, `attempt_id`. Logs are structured JSON with redaction at the logger: the phone appears masked (last four digits), names and message text never appear, and tokens are never logged. Dashboards are business-scoped and platform-wide. Restricted diagnostics that need customer fields go through the audited operator path, not telemetry (I11).

## 15.2 Service-level indicators and alerts

| Signal | Why it matters | Level |
|---|---|---|
| Webhook 5xx rate and acknowledgement p99 | Ingress availability (T-2) | Page |
| Age of the oldest unprocessed inbound event | Processing stall; order guarantee at risk | Page above 2 minutes |
| Operations `uncertain` beyond the settle window | T-6 | Page |
| Reminder occurrences due but unsubmitted 10 minutes after due, while live | T-5 | Page |
| Outbound attempts `uncertain` (count and age) | Compliance and customer experience | Ticket; page on a spike |
| Calendar `Unavailable` ratio per business and globally | Provider or credential trouble | Ticket; page if global |
| Credential-failed or authorization-revoked flags | Business cannot be served | Page |
| Template not approved for a live business | Reminders blocked | Page |
| Integrity conflicts and quarantined ingress | Tampering or misconfiguration | Ticket; page on a burst |

| Signal | Why it matters | Level |
|---|---|---|
| Database failover, replication lag, connection or disk pressure, long locks | Platform health | Page |
| Newest backup older than RPO; failed restore drill | T-4 | Page; ticket |
| Oldest unattended human handoff | F-11 | Ticket |
| Any sentinel violation (§15.4) | Invariant breach | Page |

## 15.3 Review queue and operator commands

`review_items` is the single place where work needing a human decision lands. Types: uncertain operation, uncertain send, external overlap, appointment in review, integrity conflict, poison event, unattended handoff, credential failure, template blocked. Each item links to evidence and a runbook. Illustrative CLI surface (names are not requirements, per the baseline): `status`, `review list|show|resolve`, `operation reconcile`, `message retry --reason` (the explicit U-RETRY path), `permission record`, `credential probe|rotate`, `quarantine on|off`, `restore plan|run`, `sentinels run`. Every mutating command needs a reason and is audited. Reconciliation is repeatable and never forces a release, send or write (O02).

## 15.4 Invariant sentinels

Read-only SQL checks that run hourly in production, after every automated test scenario, and after every restore. A violation is an incident.

1. No two `held` or `committed` reservations overlap on one Calendar under different appointment keys.
2. Every `verified_applied` booking has an appointment row and a committed or later-released reservation.
3. No operation sits `in_flight` with an expired lease beyond the recovery interval.
4. No outbound attempt sits `submitting` with an expired lease.
5. No message has two live attempts; no attempt follows `uncertain` without an operator link.
6. For every attempt, effective delivery rank equals the maximum rank among its receipts.

7. No reminder was accepted outside its [due, `expires_at`) window (small tolerance for clock skew).
8. No reminder was submitted for an appointment that was cancelled, past or on a superseded schedule.
9. Inbound events were processed in sequence order per conversation.
10. Parent and child rows agree on `business_id` (foreign-key backstop and lint).
11. Every business with live capabilities has a credential verified within a configured age.

### 15.5 Canary and playbooks

A canary business with its own WhatsApp number and test Calendar runs a scripted booking, move, cancel and reminder cycle on a schedule in production. It detects provider drift, template status changes and end-to-end breakage before customers do. Runbooks exist for: webhook failure, processing stall, Google outage, Meta outage, credential failure, database failover and restore (§16). Provider outages degrade gracefully: Google down means `Unavailable` replies and no writes; Meta down means webhooks stop arriving, replies queue within their expiry, and ordinary replies that expire are retained as history.

## 16. Backup, restore and disaster recovery

### 16.1 Layers

| Layer | Mechanism | Covers |
|---|---|---|
| Continuous | Managed point-in-time recovery | RPO of T-4 for database loss or corruption |
| Snapshots | Daily, retained for a period set by the owner (35 days proposed) | Longer rollback; verified before pruning (O03) |
| Ingress archive | Write-once copies of raw signed webhook bodies, 30 days proposed (T-7) | Inbound events and receipts after the restore point |
| Infrastructure | Everything as code in version control | Rebuilding the platform |
| Keys and secrets | KMS keys and the secrets manager with their own recovery runbook | Credentials (O03 requires a separate plan) |

| Layer | Mechanism | Covers |
|---|---|---|
| Residency | Cross-region copies only if the owner's residency decision permits | Regional loss |

## 16.2 What a restore cannot do

It cannot unsend a WhatsApp message and cannot undo or recreate a Google event. Provider state after the restore point is therefore **unknown to the restored database**, and the procedure treats it that way.

## 16.3 Restore procedure

1. **Restore** to a new, isolated instance at the chosen time.
2. **Start in quarantine.** Outbound sending and the reminder scheduler are off. Read-only reconciliation may run.
3. **Check integrity** by running the sentinels.
4. **Replay the ingress archive** from the restore point onward through the normal inbox. Deduplication makes this safe. Replayed customer messages older than their session are treated as stale by the reducer.
5. **Reconcile the Calendar.** List events by `nm_biz` updated since the restore point minus a margin. Events with an unknown operation are reported as orphans and adopted only by a reviewed operator action. Divergences (the database says cancelled but the event exists, and the reverse) become review items.
6. **Reconcile outbound.** Every attempt that is `prepared` or `submitting`, or created after the restore point and not otherwise known, is marked `uncertain`. Archived receipts then resolve what they can.
7. **Reminders.** Occurrences that fell due in the gap and are older than 15 minutes are skipped by the ordinary rule. Any occurrence whose attempt state is unknown is `uncertain` and is never resent.
8. **Leave quarantine business by business** after review.

Two defences hold even if reconciliation is incomplete. Booking always rechecks Google, so an event whose reservation row was lost still blocks the slot. And creation is idempotent by event ID, so a replayed booking operation cannot create a second event.

**Operator command journal.** Every operator mutation (consent evidence, configuration and capability changes, credential rotations, channel mappings) is also appended to the write-once archive as a replayable command journal. Without it, a restore would silently lose evidence recorded after the restore point. Replay happens in step 4, before quarantine ends (see Deliverable 4, scenario 10).

### 16.4 Residual risks (stated honestly)

- Ordinary replies sent after the restore point may be produced again when the inbound event is reprocessed. Duplicated ordinary replies are annoying but not harmful, and the 24-hour window bounds them. Reminders are not exposed to this (step 7).
- A customer message accepted and acknowledged after the last archived copy and the restore point is lost. Archive export latency therefore sets the loss window and is part of T-4.
- The restore drill is the only evidence the procedure works (§17).

## 17. Testing architecture

| Layer | Technique | What it proves |
|---|---|---|
| Domain | Unit tests plus property-based tests on pure modules | Reducer legality; date grammar; availability including touching intervals and DST; receipt merge commutativity; reminder boundaries at exactly 169 h, 168 h, 24 h and 2 h; label format |
| Repository | Real Postgres in containers | RLS, composite foreign keys, exclusion-constraint races, claim queries under concurrent workers |
| Provider simulators | In-process Meta and Calendar simulators with fault injection | Meta: signature handling, duplicates, reordering, delays, drops. Calendar: ETags, event-ID uniqueness, free/busy, property filters, and the faults *fail before commit*, *succeed then lose the response*, *slow*, *malformed* |

| Layer | Technique | What it proves |
|---|---|---|
| Crash points | Named crash points compiled into test builds; a runner executes a scenario in a child process, kills it at point *k*, restarts it on the same database, then asserts outcomes and sentinels | Every row of §8.4 and §11.3; I12 (a test keeps its own interrupted state across restart) |
| Concurrency | Parallel bookers on one slot; stale worker resuming after a pause | Exactly one winner; fencing prevents stale commits (I06) |
| Time | Virtual clock simulation | Restart, midnight, DST and year boundaries; 15-minute allowance (I10) |
| Isolation | Generated suite that, for every table and repository function, runs as tenant A against tenant B's rows | No cross-tenant read or write (I01) |
| Security | Signature mutation fuzzing; forged and replayed tokens; log-redaction tests; dependency audit | W01, I03, I11 |
| Recovery | Nightly small restore drill in CI; periodic production-like drill; the §16.3 procedure scripted | O03, I12 |
| Contract and drift | Recorded provider samples; a weekly job against real sandbox accounts | The simulators still match reality |
| End to end | Staging with a real test WhatsApp number and Calendar; production canary | Integration holds |

**Acceptance is traced to requirements, not counted.** Every test is tagged with baseline IDs and CI reports coverage by ID. The baseline's minimum acceptance scenario set (Section 7) maps one-to-one to scenarios in this suite. The existing 1,089 tests are evidence for the POC, not a target for this design (Q01).

**Honest limit:** simulators encode my beliefs about provider behaviour. The drift job and the real-sandbox runs exist precisely because a simulator can be confidently wrong.

## 18. Traceability summary

| Baseline requirements | Where this design satisfies them |
|---|---|
| S01, O04 (capabilities, durable mode) | §6, §12.1, §14; single durable mode (Deliverable 1 §8) |
| W01, W02 (signature, tenant by channel) | §5.1, §12.2 |
| W03–W09 (conversation, grammar, labels) | §6 |
| C01–C03 (Calendar scope, availability, unavailable) | §7.1, §7.3 |
| C04–C09 (booking, lookup, move, cancel, truth vs record) | §7.4, §8, §9 (U-REPLAY in Deliverable 1) |
| R01–R12 (reminders, attendance, permission, templates, modes) | §10, §11 |
| I01–I03 (scope, repeats, offered actions) | §6.2, §12.2 |
| I04, I05, I12, I13 (durable accept, order, recovery, expiry) | §5, §10, §16 |
| I06–I09 (concurrency, uncertainty, finalization, retries) | §4.3, §8, §9, §10.3 |
| I10 (time) | R6, §7.3, §11.1 |
| I11 (credentials, diagnostics) | §7.1, §13, §15.1 |
| O01–O03 (operator, reconciliation, backup) | §13.2, §15, §16 |
| Q01 (quality goals) | §15.4, §17; targets T-1 to T-7 in Deliverable 1 |

# 19. Design risks I am carrying

| Risk | Mitigation | Reconsider when |
|---|---|---|
| Postgres as scheduler and queue could bottleneck on claim contention | Bounded claim batches; index design; measure at 10× T-1 | Sustained claim latency, or a need for cross-service fan-out |
| Idempotent Calendar insert behaves differently than assumed (S-1) | Spike first; fallback to read-only proof plus operator resolution | S-1 fails |
| Opaque callback value is not echoed on receipts (S-6) | Correlate by recipient, window and message ID; treat more outcomes as uncertain | S-6 fails |
| Row-level security adds complexity and pooling pitfalls (S-10) | Tenant-scoped session wrapper; generated leakage tests | Performance or operability problems |
| Settle window and receipt-wait are guesses | Configurable; measure provider latency in staging and canary | First real data |
| Single region and single primary | Multi-AZ failover; tested restore; accepted outage up to T-4 | Owner sets a stricter availability target |

The full strengths, weaknesses and pre-production evidence list is in Deliverable 5.